

Dynasties Platform — main branch

Hostile-but-fair, read-only PR review of the latest origin/main on github.com/lifecoacher/dynasties-platform. No code was modified, no branches were checked out for the user's working environment, and no fixes were applied.

Repository	lifecoacher/dynasties-platform
Branch audited	main (latest push)
HEAD audited	ad9647f17db6fc7a50b11bd30a1171e23a14c32e
Expected by user	e420ce51d277b9a4e232a6bd626b322389181878
Audit date	May 3, 2026
Auditor	Perplexity Computer (acting as principal engineer / release engineer)
Mode	READ-ONLY — no fixes proposed in code, only recommendations

1 • Executive verdict

PASS WITH CONDITIONS — NOT YET PRODUCTION-READY

The platform on main is materially better than the prior baseline. The previous review's 16 hardening items are largely real: Stripe is decoupled from Replit, webhook idempotency is DB-backed, the dev role override is hard-gated behind three flags, JWT hard-fails in production, the Clerk webhook signature is properly verified, accounting sync uses real DB transactions, health and metrics endpoints exist, CI and deployment workflows are in place, and a Zod env schema enforces required variables. That is the good news.

The bad news is that two of the safety claims are partial fictions in the wired-up code, and the test suite that the CI claims to run is effectively a no-op. There is also a real database migration drift on disk: two files numbered 0003_* exist, and only one is registered in the Drizzle journal — the other will silently fail to apply. Deployments will look green and still ship a database that is missing indexes the application code expects. Combined with several smaller production-grade issues (CSP disabled, process-level error swallowing, pnpm version drift between local tooling and Docker), this is not a clean release. It is a release that needs a focused 1-2 day fix sprint before tagging.

Translation for non-technical readers: **"better than before, real progress, but do not turn this on for paying customers this week."**

2 · Git sync verdict

You asked whether the latest push on `origin/main` matches what you expected. Short answer: the SHA you expected is reachable, but it is not the tip — and the tip differs from your expected SHA by zero source files.

Expected tip (you)	e420ce5 — PR#1 merge of rc/hardening-pass-20260414
Actual tip (origin)	ad9647f — PR#2 merge of replit-agent
Source diff e420ce5 → ad9647f	0 files changed — replit-agent merge was a history rewrite, not a content change
Ancestry check	All four expected ancestor commits — feece83, 13f5fe7, 30c2bf2, 14890a4 — are reachable from origin/main
Likely cause	Secret-scrub rewrite (Anthropic API key removal — see commit e83d39c) re-merged onto main, producing a new merge commit with identical tree
Local working tree	Detached HEAD at ad9647f, clean — no modifications
Caveat	This audit cannot verify your Replit workspace — only the GitHub remote and a fresh clone in the sandbox

Verdict: the tip is functionally equivalent to the SHA you expected. You are not auditing stale code. The history rewrite around secret removal is normal hygiene and is correctly reflected on the remote.

3 · What is genuinely solid

- ✓ Stripe credential decoupling. `artifacts/api-server/src/stripeClient.ts` reads `STRIPE_SECRET_KEY` from env first, with the Replit-managed connector as secondary fallback. Production deployments will not depend on Replit.
- ✓ Webhook idempotency (DB-backed). `webhookHandlers.ts` uses `stripeWebhookEventsTable` with an INSERT-on-conflict pattern, not in-memory deduplication. Replays cannot double-charge or double-credit.
- ✓ Dev role override is hard-gated. `middlewares/auth.ts` requires `NODE_ENV !== 'production'` AND `FEATURE_DEV_ROLE_OVERRIDE=true` AND `LOCAL_DEV_ONLY=true`, and can only downgrade a role. Triple-flag gating is the right pattern.
- ✓ JWT secret hard-fails in production. `getJwtSecret()` throws when `JWT_SECRET` is missing under `NODE_ENV=production`. The Zod env schema also enforces a 32-character minimum at boot.
- ✓ Clerk webhook signature verification. `routes/clerk-webhook.ts` implements full svix-style HMAC-SHA256 verification with a 5-minute timestamp tolerance, not the placeholder you had previously.
- ✓ Accounting sync uses real transactions. `services/accounting/sync-service.ts` wraps mutations in `db.transaction(async (tx) => ...)` blocks (5+ usages). Partial sync failures will roll back.

- ✓ QuickBooks OAuth callback registered before auth router. `app.get("/api/accounting/oauth/quickbooks/callback", qbOAuthCallbackHandler)` is mounted before `authRouter`, so OAuth redirects don't get bounced by auth.
- ✓ QuickBooks token refresh wired. `RealQuickBooksAdapter.refreshAccessToken()` exists and is called on 401. The demo adapter is gated behind `VITE_DEMO_MODE=true` or `QB_MODE=demo`; default behavior is live.
- ✓ Health endpoints. `/api/healthz`, `/api/healthz/ready`, `/api/healthz/live` all present in `routes/health.ts`. ECS and load-balancer health checks have something real to talk to.
- ✓ Prometheus metrics endpoint. `/api/metrics` exposes Prometheus text format and is gated behind `METRICS_ENABLED` in production — sane default.
- ✓ CI/CD pipelines exist and are non-trivial. `.github/workflows/ci.yml` runs install + typecheck + (claimed) tests + build. `deploy.yml` builds and pushes Docker images to ECR, runs Drizzle migrations, force-redeploys ECS, syncs the frontend to S3, and invalidates CloudFront.
- ✓ Migration runner script. `scripts/run-migrations.sh` wraps `drizzle-kit migrate` and tolerates Stripe-schema migration failure. Reasonable.
- ✓ Zod env schema. `lib/config/src/env.ts` validates every required variable at boot and dumps a human-readable error block on failure. The boot will not silently start in a broken state.
- ✓ Helmet middleware present. Sets HSTS in production, disables COEP. (CSP is disabled — see Major issues.)
- ✓ Webhook signature has a safe production fallback policy. Stripe webhook verification falls back to no-verification only when `NODE_ENV !== 'production'`; throws otherwise.
- ✓ Terraform infra-as-code. `infra/main.tf` defines a real VPC with public/private subnets, NAT gateway, RDS Postgres 16.4 with encrypted storage, and security groups with proper ingress chaining (ALB → ECS → RDS). Not a toy.
- ✓ Tech debt markers are clean. 0 `TODO/FIXME/HACK/XXX` markers and 0 `@ts-ignore/@ts-expect-error` across the source tree.

4 · CRITICAL issues

These will break production or hide breakage. Do not tag a release until each is resolved.

CRITICAL

Migration drift — orphaned 0003_recommendation_hardening.sql will never run

lib/db/migrations/0003_recommendation_hardening.sql · lib/db/migrations/meta/_journal.json

Issue	Two migration files share the 0003_ prefix on disk: 0003_recommendation_hardening.sql and 0003_romantic_lester.sql. The Drizzle journal (meta/_journal.json) only references 0003_romantic_lester. drizzle-kit migrate uses the journal as the source of truth, so 0003_recommendation_hardening.sql will not be applied — silently. The orphaned migration adds the fingerprint and superseded_by_id columns and a unique partial index (recommendations_active_fingerprint_idx) used by the recommendation engine.
Why it matters	Production will deploy without these columns/index. Any code path that selects on or inserts with the fingerprint column will throw a runtime SQL error. The CI pipeline will look green because it is running against a fresh DB that doesn't include them either.
Recommended fix	Either rename 0003_recommendation_hardening.sql to 0005_recommendation_hardening.sql and add a corresponding entry to meta/_journal.json with a fresh snapshot, or fold the contents into the next clean migration generated by drizzle-kit generate. Do not hand-edit the journal — re-generate.

CRITICAL

CI test job is a silent no-op — pnpm test --if-present matches nothing

.github/workflows/ci.yml (line ~85) · package.json (root) · all artifact and lib package.json files

Issue	The CI workflow runs pnpm test --if-present. A grep across every package.json in the monorepo (root, artifacts, lib, services) returns zero packages with a test script defined. The test files in tests/money-path/, tests/beta-path/, tests/rls-enforcement/ are real (1,313 lines total) but they are never executed in CI.
Why it matters	Every PR is being merged with the green checkmark on a step that does nothing. The money-path test in tests/money-path/money-path.test.ts uses Vitest's describe/it/expect API but no Vitest config or runner is wired up. Regressions in invoice creation, billing, RLS enforcement, and the beta path will not be caught by CI.
Recommended fix	Add a root test script that runs vitest run across tests/**/*.test.ts; install vitest as a root devDependency; create a vitest.config.ts. For the integration tests that hit a live API (http://localhost:8080/api), spin up the api-server in CI as a service container or skip them with a describe.skipIf(!process.env.E2E) guard so unit tests still run.

CRITICAL**Process-level error handlers swallow uncaught exceptions and unhandled rejections**

artifacts/api-server/src/index.ts (lines 9-15)

Issue	The entrypoint installs both <code>process.on('unhandledRejection', ...)</code> and <code>process.on('uncaughtException', ...)</code> handlers that print 'server will NOT crash' and continue. Node.js documentation explicitly states this is unsafe — after an uncaught exception the process is in an undefined state.
Why it matters	A bug that should crash a worker and let ECS replace it instead silently corrupts subsequent requests. Memory leaks, half-completed DB transactions, and partially closed sockets accumulate. Customers see weird intermittent behavior that no logs explain because the logs say 'will NOT crash' rather than the actual stack. This actively defeats the value of running on ECS with a healthy restart policy.
Recommended fix	Log the error with full stack and request context, attempt a graceful shutdown of in-flight HTTP and DB work with a timeout (10-30s), then call <code>process.exit(1)</code> . Let ECS do its job. Keep the handlers — but make them terminate after logging. <code>unhandledRejection</code> should be treated the same way.

5 • MAJOR issues

These will not break the first deploy but will degrade security or operability under real traffic.

MAJOR**Helmet runs with CSP and COEP disabled**

artifacts/api-server/src/app.ts (lines 27-32)

Issue	Helmet is configured with <code>contentSecurityPolicy: false</code> and <code>crossOriginEmbedderPolicy: false</code> . CSP is the single most effective defense against XSS-class attacks; disabling it leaves the API responses without a Content-Security-Policy header. If the API serves any HTML (error pages, OAuth callback pages, or webhook debug surfaces), they have no XSS protection.
Why it matters	An XSS in any of the OAuth/webhook flows that returns HTML, or in any rendered error template, becomes exploitable when it would otherwise be blocked. CSP is also a compliance signal in security questionnaires.
Recommended fix	Define a strict CSP that allows only your own origins for <code>connect-src/frame-src</code> and Stripe/Clerk/QuickBooks for the specific endpoints that need them. The frontend is served from CloudFront/S3 separately, so the API CSP can be very tight: <code>default-src 'none'; frame-ancestors 'none'</code> .

MAJOR

CORS allows all origins in non-production — and depends on NODE_ENV being correctly set

artifacts/api-server/src/app.ts (lines 36-44)

Issue	<code>cors({ origin: isProduction ? ALLOWED_ORIGINS : true, credentials: true })</code> . If <code>NODE_ENV</code> is unset, missing, or typo'd ('Production', 'prod') the server falls into the permissive branch and reflects any origin with credentials. Combined with <code>credentials: true</code> this is a CSRF risk.
Why it matters	A misconfigured ECS task definition or a Replit-only env setting that reaches production accidentally exposes the API to credentialed requests from any site. The Zod env schema defaults <code>NODE_ENV</code> to 'development', so a missing var quietly opens this door.
Recommended fix	Invert the default: require <code>ALLOWED_ORIGINS</code> always, refuse to start if empty, and use an explicit list including localhost for dev. Or check <code>isProduction !ALLOWED_ORIGINS.length</code> and reject. Never reflect arbitrary origins with <code>credentials: true</code> .

MAJOR

docker-compose.yml ships with a default JWT secret

docker-compose.yml (line ~38)

Issue	<code>JWT_SECRET: '\${JWT_SECRET:-dev-jwt-secret-change-in-production-min32chars}'</code> — if a developer or operator runs <code>docker compose up</code> without setting <code>JWT_SECRET</code> , the API boots happily with a known, public string and <code>NODE_ENV=production</code> set in the same file.
Why it matters	The compose file pins <code>NODE_ENV: production</code> . The Zod schema enforces a 32-char minimum but does not check for known-bad values. Anyone running the stack locally for QA is signing tokens with a string that is in your public repo. If they then expose the port (or share the same DB schema) tokens become forgeable.
Recommended fix	Remove the default fallback and let the env schema fail to boot. Or add a deny-list in <code>lib/config/src/env.ts</code> that rejects any JWT secret containing the substrings 'dev-', 'change', or 'example'.

MAJOR

Structured logging is half-wired — pino registered but the request logger uses console.log

lib/config/src/logger.ts · artifacts/api-server/src/middlewares/request-logger.ts · 164 console.* calls in api-server

Issue	lib/config/src/logger.ts exports a createLogger() based on pino. Nothing in the api-server actually imports it. middlewares/request-logger.ts emits JSON via raw console.log/warn/error. Across artifacts/api-server/src there are 164 console.* calls. The prior review's claim of 'structured logging' is technically true (the request logger emits a JSON shape) but is not the pino-based correlation-id-rich logging that was claimed.
Why it matters	Log levels can't be filtered at runtime; pino's redaction (which protects PII like emails and Stripe IDs from logs) is unused; child loggers per request can't be created; integration with Datadog/CloudWatch via pino transports is missing. And every console.error in production is unstructured noise on stderr — fine for ECS log capture, bad for queryability.
Recommended fix	Replace console.* in request-logger.ts and the rest of the api-server with the createLogger() instance. Attach a per-request child logger with the x-request-id in req.locals.logger. Configure pino redaction for req.headers.authorization, email, stripeCustomerId, etc.

MAJOR

Dockerfile uses pnpm@9; root package.json declares pnpm@10.26.1

Dockerfile · Dockerfile.frontend · package.json (root) · .github/workflows/ci.yml

Issue	Dockerfile and Dockerfile.frontend hard-code corepack prepare pnpm@9 --activate. Root package.json declares packageManager: 'pnpm@10.26.1'. CI workflow uses pnpm/action-setup@v4 without pinning, which respects the packageManager field (so CI uses 10.x), while the production container build uses 9.x.
Why it matters	Lockfile incompatibilities between pnpm 9 and 10 produce different node_modules graphs in production vs. CI. Builds that pass typecheck/test in CI can fail at runtime because the resolved package versions differ. This is the single most common 'works in CI, breaks in prod' class of bug.
Recommended fix	Pin the same pnpm version everywhere: change Dockerfiles to corepack prepare pnpm@10.26.1 --activate (matching root packageManager), and pin pnpm/action-setup@v4 with version: 10 in CI. The root package.json should be the single source of truth.

6 · MEDIUM issues

MEDIUM

Login limiter set to 200/15min in non-production — high enough to mask brute force

artifacts/api-server/src/middlewares/rate-limit.ts

Issue	<code>loginLimiter</code> uses <code>max: isDevMode ? 200 : 15</code> . The dev limit of 200 is fine for local development, but the env detection is the same brittle <code>NODE_ENV !== 'production'</code> check. A staging environment that doesn't set <code>NODE_ENV=production</code> gets the dev limit.
Why it matters	If staging is internet-reachable or shares a hostname pattern with production, the loose limit allows password spraying. The middleware also disables IP validation (<code>validate: { ip: false }</code>) which silences express-rate-limit's warnings about trust-proxy correctness — useful but easy to leave on accidentally.
Recommended fix	Use a positive list: <code>['production', 'staging'].includes(NODE_ENV)</code> applies the strict limit. Add a tighter per-IP+per-account limiter (e.g. 5/15min per email) on top of the per-IP limiter.

MEDIUM

Stripe webhook handler logs to `console.error` on signature failure but returns generic 400

artifacts/api-server/src/app.ts (lines 47-72)

Issue	The inline Stripe webhook handler logs <code>'Webhook processing error'</code> via <code>console.error</code> and returns the same body for both signature-verification failure and downstream <code>processWebhook</code> exceptions. Stripe will retry 400s, so a downstream bug in webhook processing becomes an infinite retry loop with the same generic error in logs.
Why it matters	You'll have a hard time differentiating 'attacker sent us garbage' from 'our DB is down' under load. Stripe's retry backoff will hammer a service that's already failing.
Recommended fix	Distinguish signature-failure (return 400, log warn) from internal exception (return 500, log error with stack and event ID, let Stripe back off). Capture the event id from the verified payload and include it in every log line.

MEDIUM

Migration runner uses npx drizzle-kit migrate against a working directory it cd's into

scripts/run-migrations.sh · .github/workflows/deploy.yml (Run database migrations step)

Issue	scripts/run-migrations.sh does <code>cd "\$(dirname "\$0")/../../lib/db"</code> and then <code>npx drizzle-kit migrate</code> . The deploy workflow does <code>cd lib/db && npx drizzle-kit migrate</code> directly (does not call the script). They'll behave identically today but they are two separate code paths to maintain.
Why it matters	When you change the migration command (e.g. add a lock file check, switch to <code>migrate-up</code> , add a structured-output flag) you'll have to remember both. One will drift.
Recommended fix	Have <code>deploy.yml</code> call <code>scripts/run-migrations.sh</code> directly. Single source of truth.

MEDIUM

Anthropic API key history rewrite is correct hygiene, but no secret-scanning gate is in CI

.github/workflows/ci.yml · git history (commit e83d39c)

Issue	Commit e83d39c ('security: remove Anthropic API key from tracked files') was force-pushed and the replit-agent merge that landed on main was history-rewritten to scrub it. Good. But <code>ci.yml</code> has no pre-merge secret scanner (gitleaks, trufflehog, github-advanced-security secret scanning).
Why it matters	The next API key leak will reach main again before someone notices. Recovery requires a force push, which you've now done at least once and which is operationally expensive.
Recommended fix	Add a gitleaks step to <code>ci.yml</code> that runs on PRs. Enable GitHub native Push Protection on the repo (Settings → Code security). Rotate the leaked Anthropic key immediately if it has not been already.

MEDIUM

request-logger.ts emits JSON to stdout but doesn't include user/tenant context

artifacts/api-server/src/middlewares/request-logger.ts

Issue	Each request gets an <code>x-request-id</code> UUID and the JSON log line includes <code>method/path/status/duration</code> . It does not include <code>userId</code> , <code>companyId</code> , or <code>orgId</code> — fields populated downstream by the auth middleware.
Why it matters	Incident triage requires correlating a customer report ('I saw an error at 3:14pm') to logs. Without <code>companyId</code> in every log line, you cannot filter to a single tenant in CloudWatch/Datadog.
Recommended fix	Capture <code>req.user?.id</code> and <code>req.user?.companyId</code> in the response 'finish' event after auth has populated them. Log them on every line. Pino child loggers (see Major issue) make this trivial.

MEDIUM

Demo data leakage path — DEMO_MESSAGE_ID hard-coded in demo route

artifacts/api-server/src/routes/demo.ts (line 28)

Issue	The demo router contains a hard-coded message id '<demo-bo1-2026-03-12@megatech-sz.cn>' and queries ingestedEmailsTable by that id. Whether this code path is reachable in production depends on whether app.use('/api', demoRouter) is gated by env — and reading app.ts shows it is mounted unconditionally.
Why it matters	If the demo router exposes any state-mutating endpoints, customers can poke them in production. Even if read-only, demo endpoints reveal internal schema and id formats.
Recommended fix	Mount demoRouter behind if (process.env.VITE_DEMO_MODE === 'true' env.NODE_ENV !== 'production'). Or split it into a separate service that only runs in non-prod. Audit each demo route for write side-effects before deciding.

7 • MINOR issues / cleanup

- 164 `console.*` calls in `artifacts/api-server/src`. Migrate to the pino logger as part of the Major-level structured-logging fix; no individual call is dangerous.
- `app.set('trust proxy', 1)` is correct for ALB but not configurable. If you ever put another proxy in front (Cloudflare → ALB → ECS) the value will be wrong. Pull from `process.env.TRUST_PROXY` with a default of 1.
- `infra/main.tf` uses Terraform S3 backend without DynamoDB lock table. State file corruption risk under concurrent `terraform apply`. Add a `dynamodb_table` for state locking.
- Deploy workflow runs migrations before the new image is rolled out. Correct order to avoid 'app expects column that doesn't exist'. But there is no schema-compatibility check for the old running image against the new schema. Use expand-then-contract migrations or guard with feature flags.
- CI workflow caches `node_modules` per arch but uses `actions/cache/restore` with no fallback key. First-time miss on a new branch will pay full install cost. Acceptable; just slow.
- `scripts/run-migrations.sh` uses `set -euo pipefail` but the Stripe step is wrapped in `|| true`. Correct intent — Stripe migration failure should not abort drizzle migrations — but the swallowed error should be logged structurally.
- CORS allowed headers list omits `x-clerk-*`. If Clerk's hosted UI ever sends custom headers, browser preflight will fail. Currently webhooks are server-to-server, so this is theoretical.
- Frontend Dockerfile copies `nginx.conf` without a fallback. If `nginx.conf` is ever missing the build will fail with a confusing `COPY` error rather than a clear missing-file message. Add a `RUN ls -la` sanity step or a fallback.
- Stripe webhook signature timestamp tolerance is hard-coded inside webhook handler. Document the tolerance or expose via `env`. Defenders auditing this six months from now will have to re-derive what window is used.

8 • Claimed fixes — one-line re-verification

Claim	Status	Evidence
Stripe credential decoupling	VERIFIED	<code>stripeClient.ts</code> reads <code>STRIPE_SECRET_KEY</code> env first, Replit fallback secondary.
Webhook idempotency (DB-backed)	VERIFIED	<code>stripeWebhookEventsTable</code> <code>INSERT-on-conflict</code> in <code>webhookHandlers.ts</code> .
Dev role override hard-gated	VERIFIED	<code>auth.ts</code> requires <code>NODE_ENV !== production</code> AND <code>FEATURE_DEV_ROLE_OVERRIDE=true</code> AND <code>LOCAL_DEV_ONLY=true</code> ; can only downgrade.
Helmet middleware	PARTIAL	Helmet is on, but <code>contentSecurityPolicy: false</code> and <code>crossOriginEmbedderPolicy: false</code> . See Major #1.
Accounting sync transactions	VERIFIED	<code>5+ db.transaction(async (tx) => ...)</code> blocks in <code>sync-service.ts</code> .

Claim	Status	Evidence
Clerk webhook signature	VERIFIED	svix-style HMAC-SHA256 with 5-min timestamp window in routes/clerk-webhook.ts.
Health endpoints	VERIFIED	/api/healthz, /api/healthz/ready, /api/healthz/live in routes/health.ts.
CI/CD pipeline	VERIFIED	ci.yml + deploy.yml exist; deploy.yml ships ECR + ECS + CloudFront.
Migration runner	VERIFIED	scripts/run-migrations.sh wraps drizzle-kit migrate with Stripe schema migrations.
JWT hard-fail in production	VERIFIED	getJwtSecret() throws if missing in production; Zod env schema enforces 32-char minimum.
QB OAuth callback before auth	VERIFIED	qbOAuthCallbackHandler mounted before authRouter in app.ts.
QB token refresh	VERIFIED	RealQuickBooksAdapter.refreshAccessToken() implemented; called on 401.
Prometheus metrics endpoint	VERIFIED	/api/metrics with full Prometheus exposition format; gated behind METRICS_ENABLED in prod.
Request correlation IDs / structured logging	PARTIAL	x-request-id UUID is added and JSON is emitted, but via console.* not pino. See Major #4.
Zod env schema	VERIFIED	lib/config/src/env.ts validates all required vars with prettifyError boot-time output.
Webhook signature production fallback	VERIFIED	Falls back to no-verification only when NODE_ENV !== production; throws otherwise.

Score: 14 of 16 claimed fixes are fully verified. 2 are partially implemented and overstated.

9 · Release readiness

NOT READY NOW — READY WITH FIXES (1-2 day sprint)

There are no blockers that require a full re-architecture. The shape of the codebase is right: real CI, real deploy pipeline, real infra-as-code, real auth, real webhook hardening, real DB transactions. The pieces that are wrong are concentrated and fixable.

Must fix before tagging a release:

- Resolve the migration drift (orphaned `0003_recommendation_hardening.sql`) — Critical #1.
- Wire up the Vitest test runner so CI actually executes the test files in `tests/` — Critical #2.
- Replace the swallowing `uncaughtException/unhandledRejection` handlers with `log-and-exit` — Critical #3.
- Pin `pnpm` version in `Dockerfiles` to match `root packageManager` field — Major #5.
- Remove the default `JWT_SECRET` fallback from `docker-compose.yml` — Major #3.
- Tighten CORS to a positive allow-list with no permissive default — Major #2.
- Enable a strict CSP for the API — Major #1.

Should fix in the same sprint:

- Migrate from `console.*` to the registered `pino` logger — Major #4.
- Gate the demo router behind an `env` flag — Medium #6.
- Add `gitleaks/secret-scan` to CI — Medium #4.
- Add tenant context (`userId`, `companyId`) to every request log — Medium #5.
- Stripe webhook handler — distinguish 4xx vs 5xx — Medium #2.

Can land in a follow-up:

- Terraform DynamoDB state lock.
- Per-account login limiter on top of per-IP.
- Document Stripe timestamp tolerance and expose via `env`.
- `TRUST_PROXY` as configurable `env` var.
- Consolidate CI migration step to call `scripts/run-migrations.sh`.

10 · Next best action

Open a single PR titled 'release/1.0 — production gate fixes' from a fresh branch off `main`. The PR description should reference each finding by its severity tag and id from this report. The PR should contain, in order:

- Re-generate the migration: `delete 0003_recommendation_hardening.sql`, run `pnpm --filter @workspace/db run generate`, commit the resulting `0005_*` file plus journal entry. Verify on a scratch DB.
- Add `vitest` + `vitest.config.ts`; add a root-level test script; tag E2E-only specs with `describe.skipIf`; verify CI run shows real test count, not zero.
- Rewrite the `uncaughtException/unhandledRejection` handlers to `log+drain+exit(1)`.

- Pin `pnpm@10.26.1` in both Dockerfiles. Re-build images locally and verify the lockfile resolves identically.
- Drop the JWT default in `docker-compose.yml`; add a known-bad-substring check in the Zod env schema.
- Tighten CORS — explicit allow-list, refuse to start if empty in production.
- Add a strict CSP via Helmet — start with `default-src 'none'; frame-ancestors 'none'` on the API.
- Replace `console.*` in `request-logger.ts` and the rest of the api-server with the registered pino logger; add request-scoped child loggers.
- Add a `gitleaks` action to `ci.yml`; enable GitHub Push Protection on the repo.
- Once the above lands and is reviewed, tag `v1.0.0` and let the deploy workflow ship it.

Estimated cost of the above: 1-2 engineer-days for an experienced TypeScript/DevOps generalist. Most of the changes are config and small refactors; the only one with non-trivial scope is the pino migration, which can be done iteratively without blocking the others.

Audit conducted in read-only mode. No commands ran against the GitHub remote that altered state. Local clone in the audit sandbox is at detached HEAD `ad9647f`, working tree clean. The user's Replit workspace was not inspected — verification was limited to `origin/main` as published on GitHub.